Cake Pattern

ke · Multi-layer repo→service→controller app; test cake swaps fak · Build with Wa

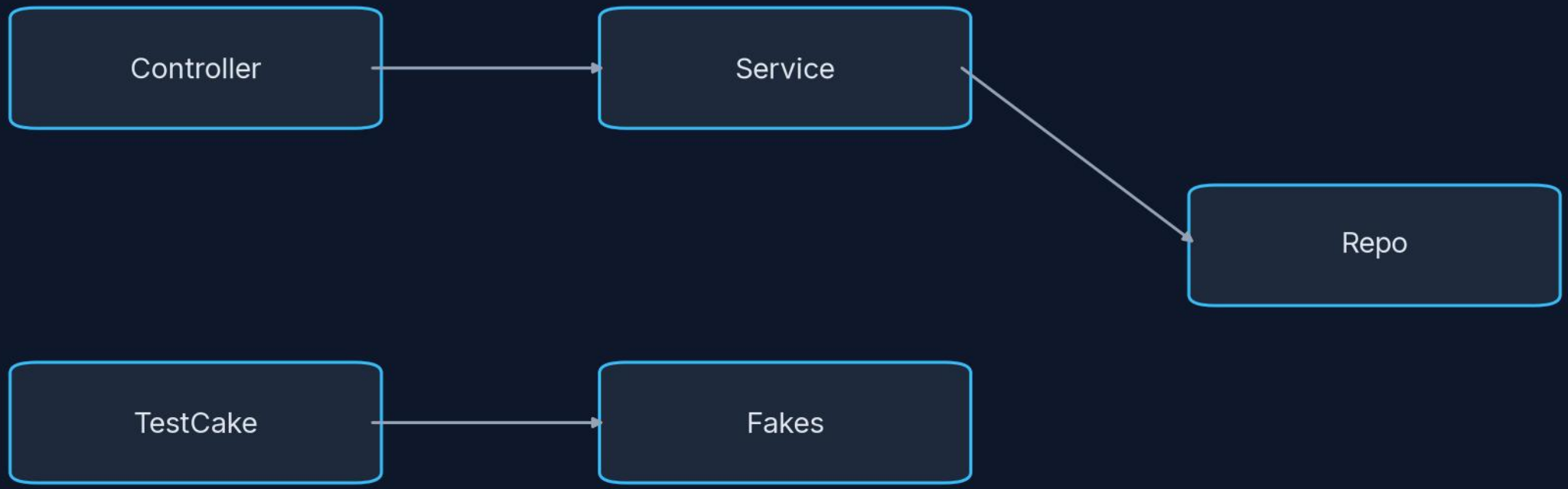
Today's Hook

- Today's force: Multi-layer repo→service→controller app; test cake swaps fakes
- We earn Cake by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: AppCake, User, UserRepo, EmailGateway

The Problem

- HTTP-ish controller → service → repo for user register/welcome.
- LiveCake wires real Map DB + mailbox; TestCake seeds fakes.
- Naive controller news everything — cannot swap email gateway.
- Success: 201 on register, 200 welcome, 409 duplicate, 404 missing — mail recorded.

Structure



Cake: Multi-layer repo→service→controller app; test cake

Real-World Scenario

- Running example: Multi-layer repo→service→controller app; test cake swaps fakes
- Key types: AppCake, User, UserRepo, EmailGateway
- Happy-path stdout cue: 201:u1
- Failure-path stdout cue: 409:exists:u2

Naive Code

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

import java.util.LinkedHashMap;
import java.util.Map;

/** Naive: controller news service news repo — cannot swap fakes without rewriting. */
public final class NaiveApp {
    private final Map<String, String> db = new LinkedHashMap<>();
    public String postRegister(String id, String email) {
        if (db.containsKey(id)) return "409:exists";
        db.put(id, email);
        System.out.println("email " + email); // hard-wired
        return "201:" + id;
    }
}
```

Why Naive Hurts

- NaiveApp.postRegister hard-wires System.out email.
- No seam to inject a fake mailbox in unit tests.
- Controller, service, and repo responsibilities collapse into one class.
- Duplicate handling returns a stringly '409' without shared types.

Solution

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Multi-layer cake: Repo → Service → Controller, with a test cake swapping fakes. */
public final class AppCake {
    private AppCake() {}

    public record User(String id, String email) {}

    public interface UserRepo {
        Optional<User> find(String id);
        User save(User user);
    }

    public interface EmailGateway {
        void send(String to, String body);
        List<String> sent();
    }
```

Solution (2)

```
public interface UserService {
    User register(String id, String email);
    String welcome(String id);
}

public interface UserController {
    String postRegister(String id, String email);
    String getWelcome(String id);
}

public interface UserRepoComponent { UserRepo userRepo(); }
public interface EmailComponent { EmailGateway emailGateway(); }
public interface UserServiceComponent extends UserRepoComponent, EmailComponent {
    default UserService userService() {
        return new UserService() {
            public User register(String id, String email) {
                var existing = userRepo().find(id);
                if (existing.isPresent()) throw new IllegalStateException("exists:" + id);
                var u = userRepo().save(new User(id, email));
                emailGateway().send(email, "welcome " + id);
                return u;
            }
```

Solution (3)

```
            public String welcome(String id) {
                return userRepo().find(id)
                        .map(u -> "hello " + u.email())
                        .orElseThrow(() -> new IllegalArgumentException("missing:" + id));
            }
        };
    }
}
public interface UserControllerComponent extends UserServiceComponent {
    default UserController userController() {
        return new UserController() {
            public String postRegister(String id, String email) {
                try {
                    var u = userService().register(id, email);
                    return "201:" + u.id();
                } catch (IllegalStateException e) {
                    return "409:" + e.getMessage();
                }
            }
            public String getWelcome(String id) {
                try {
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Cake: repo → service → controller ===");
        System.out.println("--- naive ---");
        System.out.println(new NaiveApp().postRegister("u1", "a@b.c"));

        System.out.println("--- live cake ---");
        var live = new AppCake.LiveCake();
        System.out.println(live.userController().postRegister("u2", "x@y.z"));
        System.out.println(live.userController().getWelcome("u2"));
        System.out.println("mail=" + live.mailbox());
        System.out.println(live.userController().postRegister("u2", "x@y.z")); // conflict

        System.out.println("--- test cake with fakes ---");
        var test = new AppCake.TestCake().seed(new AppCake.User("seed", "s@e.ed"));
        System.out.println(test.userController().getWelcome("seed"));
        System.out.println(test.userController().postRegister("u3", "t@est.io"));
        System.out.println("mail=" + test.mailbox());
        System.out.println(test.userController().getWelcome("missing"));
    }
```

Runnable main (2)

```
}
```

Walkthrough Output

```
=== Cake: repo → service → controller ===
--- naive ---
email a@b.c
201:u1
--- live cake ---
201:u2
200:hello x@y.z
mail=[x@y.z:welcome u2]
409:exists:u2
--- test cake with fakes ---
200:hello s@e.ed
201:u3
mail=[t@est.io:welcome u3]
404:missing:missing
```

Step-by-Step Trace

- Match stdout to stages — look for: 201:u1
- Then the injected failure: 409:exists:u2
- Re-run: java -cp ... 60-cake-pattern ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.functional.cake.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** Multi-layer cake: Repo → Service → Controller, with a test cake swapping fakes. */
public final class AppCake {
    private AppCake() {}

    public record User(String id, String email) {}

    public interface UserRepo {
        Optional<User> find(String id);
        User save(User user);
    }

    public interface EmailGateway {
        void send(String to, String body);
        List<String> sent();
    }
```

Modern Java 25 / Zazr (2)

```
public interface UserService {
    User register(String id, String email);
    String welcome(String id);
}

public interface UserController {
    String postRegister(String id, String email);
    String getWelcome(String id);
}

public interface UserRepoComponent { UserRepo userRepo(); }
public interface EmailComponent { EmailGateway emailGateway(); }
public interface UserServiceComponent extends UserRepoComponent, EmailComponent {
    default UserService userService() {
        return new UserService() {
            public
```

Trade-offs

- Use Cake when «Multi-layer repo→service→controller app; test cake swaps fakes» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Cake in product language.

Related Patterns

- Name the pattern usually confused with Cake and separate the forces.
- Sketch the roles for «Multi-layer repo→service→controller app; test cake swaps fakes» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Multi-layer repo→service→controller app; test cake swaps fakes
- Naive pain → Cake boundary → Demo stdout.
- Remember the failure cue: 409:exists:u2
- Break a ProblemTest assertion, watch it fail, restore it.